

Production Ready AI Automation Revenue Engine

Executive summary

The right way to build this system is **not** as a password-replay bot that logs into arbitrary websites and social systems with raw credentials. A production-ready implementation should treat the website CMS and social platforms as **connector targets**, detect what is actually present, prefer **official OAuth or platform APIs**, store secrets in a centralized vault, rotate machine secrets automatically, and only use direct admin credentials where the target platform explicitly supports secure API or app-password style access. That conclusion is strongly supported by official security guidance and by current platform docs from HubSpot, X, Pinterest, Streamlit, FastAPI, OWASP, and NIST. ¹

For LLM connectivity, the safest design is a **provider adapter layer** with a normalized interface for chat, embeddings, reranking, image generation, safety checks, and analytics enrichment. Current official docs support a strong baseline for OpenAI, xAI, Gemini, Anthropic, and NVIDIA NIM, but they are **not all wire-compatible** in exactly the same way. OpenAI documents the Responses API, authentication headers, and embeddings; xAI explicitly documents an OpenAI-compatible base URL; Gemini documents structured outputs, function calling, long context, file search, and an OpenAI compatibility guide; NVIDIA's current docs show that **LLM chat** and **embeddings** are on `integrate.api.nvidia.com`, while at least one current image-generation endpoint is documented under `ai.api.nvidia.com/v1/genai/...`, which means an older assumption that "all NIM endpoints use one base URL" is no longer universally safe. ²

On NVIDIA specifically, the official catalog has grown beyond the "77 free models" framing in your earlier materials. The current official models page spans **LLM, retrieval, visual, multimodal, healthcare, route optimization, and climate simulation** sections; counting the model entries visible on the current page yields a much larger catalog, so a fixed "77 endpoint JSON" should be treated as **stale or non-authoritative** unless revalidated live at build time. NVIDIA's build portal does currently advertise **free serverless APIs for development** and shows featured cards such as **nemotron-3-ultra-550b-a55b**, **kimi-k2.6**, and **glm-5.1** with "Free Endpoint" labeling, but that availability should be re-checked per model card during implementation. ³

The best implementation pattern for your role as a Director of Sales & Marketing is a **Streamlit control plane** for admin-friendly operation, backed by a **FastAPI service layer** that handles authentication, encrypted credential storage, provider routing, lead ingestion, CRM synchronization, campaign orchestration, experiment tracking, and retrieval-augmented prompting. Streamlit officially supports secrets management, theming, and app testing; FastAPI officially documents JWT-based auth and first-class testing; OWASP and NIST support centralized secret management, automated machine-secret rotation, least-privilege access, long passwords for password-only flows, and avoiding arbitrary forced password changes for human users. ⁴

What follows gives you the **single master prompt**, the target architecture, a runnable Streamlit/FastAPI scaffold, deployment guidance, testing patterns, a realistic low-end infrastructure estimate, and a milestone

checklist. Where your specification is still open—CMS type, hosting target, exact social platforms, exact CRM, and budget—I note that explicitly instead of guessing.

Scope and non negotiable design constraints

Several of your requirements are still intentionally open, and the prompt should force the AI agent to preserve those unknowns instead of inventing them. The major unspecified items are the **CMS type, hosting environment, social platforms, CRM system, analytics stack, payment stack**, and the exact project inventory beyond “100+ projects available.” That means the agent should implement a **connector discovery and adapter strategy**, not a single hard-coded CMS or social integration.

The most important product decision is how to interpret “provide website link, admin credentials, and social media login credentials.” In a secure production design, those inputs should be converted as quickly as possible into **platform-native tokens, app credentials, scoped OAuth grants, or app-specific passwords**. HubSpot explicitly states that OAuth uses authorization tokens rather than passwords. X documents app credentials, OAuth 1.0a, OAuth 2.0, callback requirements, exact redirect matching, secure storage, credential rotation, separate apps for dev/staging/prod, and least-privilege permissioning. Pinterest likewise requires app registration, redirect URI exact matching, and OAuth 2 authorization flow. That evidence supports a hard prompt rule: **use raw passwords only for one-time verification or where the platform explicitly requires them; otherwise migrate to scoped tokens immediately.** 5

The website side should be treated similarly. Because the CMS is unspecified, the system should first perform low-risk discovery—homepage fingerprinting, robots/sitemap retrieval, form detection, analytics tag discovery, DNS or HTML ownership verification where possible, and admin endpoint pattern detection. Only after platform identification should the system request the minimum viable integration path. This is an architectural inference from the least-privilege and centralized secret-management guidance in OWASP and from the platform token models above. 6

The revenue engine itself should be organized around a few outcome-oriented modules instead of a feature dump: **lead capture, qualification, CRM sync, outreach sequencing, landing-page and campaign experiments, analytics attribution, content and ad creative generation, knowledge retrieval, and monetization workflow orchestration**. That structure maps cleanly onto Streamlit as an operator console and FastAPI as the execution backend. Streamlit’s documented secrets, theme, config, and testing support make it suitable for the control surface; FastAPI’s JWT and testing patterns make it suitable for the execution layer. 7

Current provider and connector research

The table below consolidates the highest-confidence, official findings you can safely bake into the architecture.

Provider or platform	What the current official docs support	Practical implication for your engine
OpenAI	Bearer authentication, optional organization and project headers, Responses API, embeddings, production best-practice guidance. ⁸	Use as a first-class text, reasoning, and embedding provider behind a <code>ProviderAdapter</code> .
Anthropic	Current official getting-started and API onboarding flow for Claude. ⁹	Add as a provider-specific adapter, not as an assumed OpenAI drop-in.
Gemini	Structured outputs, function calling, long context, File Search, Live API, streaming, OpenAI compatibility guidance. ¹⁰	Strong fit for multimodal and agent workflows, but keep a dedicated adapter path.
xAI	API key creation, Responses API examples, and an OpenAI-compatible base URL at <code>https://api.x.ai/v1</code> . ¹¹	Easy to normalize with an OpenAI-style provider interface.
NVIDIA NIM	LLM overview at <code>https://integrate.api.nvidia.com</code> with <code>POST /v1/chat/completions</code> ; embedding endpoint documented at <code>https://integrate.api.nvidia.com/v1/embeddings</code> ; some image-generation pages use <code>https://ai.api.nvidia.com/v1/genai/...</code> ; build portal offers API keys and free serverless APIs for development. ¹²	Do not assume one base URL for every modality. Separate LLM, embedding, and image adapters.
HubSpot	Latest docs use date-versioned APIs under <code>https://api.hubapi.com/</code> ; OAuth uses tokens rather than passwords; installation requires scopes and an app. ¹³	Best default CRM adapter for a sales-and-marketing engine when CRM is unspecified.
X	App credentials, OAuth 1.0a and OAuth 2.0, bearer tokens, callback URL requirements, credential rotation, least privilege, separate apps by environment. ¹⁴	Use for owned account publishing, analytics pull, and safe automation.
Pinterest	App registration, redirect URI exact matching, OAuth 2 flow, app ID and secret. ¹⁵	Support as a tokenized social connector instead of raw password automation.
Streamlit	<code>st.secrets</code> , <code>.streamlit/secrets.toml</code> , <code>.streamlit/config.toml</code> , theming, cookie secret for multi-replica consistency, testing via <code>AppTest</code> . ¹⁶	Ideal for the operator UI and easy admin configuration.

Provider or platform	What the current official docs support	Practical implication for your engine
FastAPI	JWT auth tutorial, strong testing support via <code>TestClient</code> , pytest-friendly design. ¹⁷	Ideal for modular backend endpoints and service orchestration.

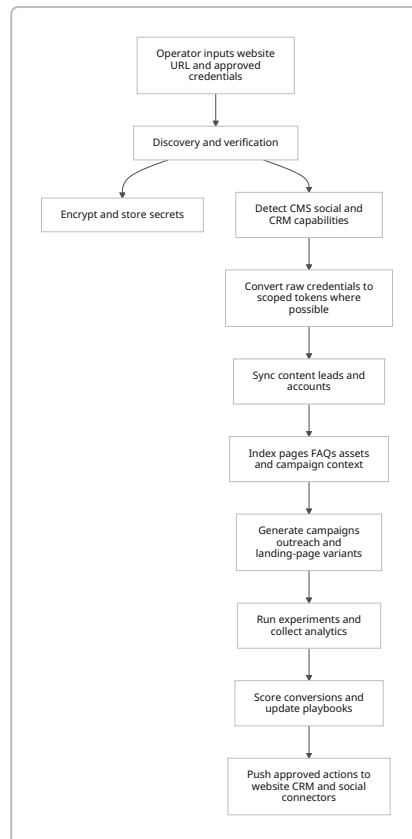
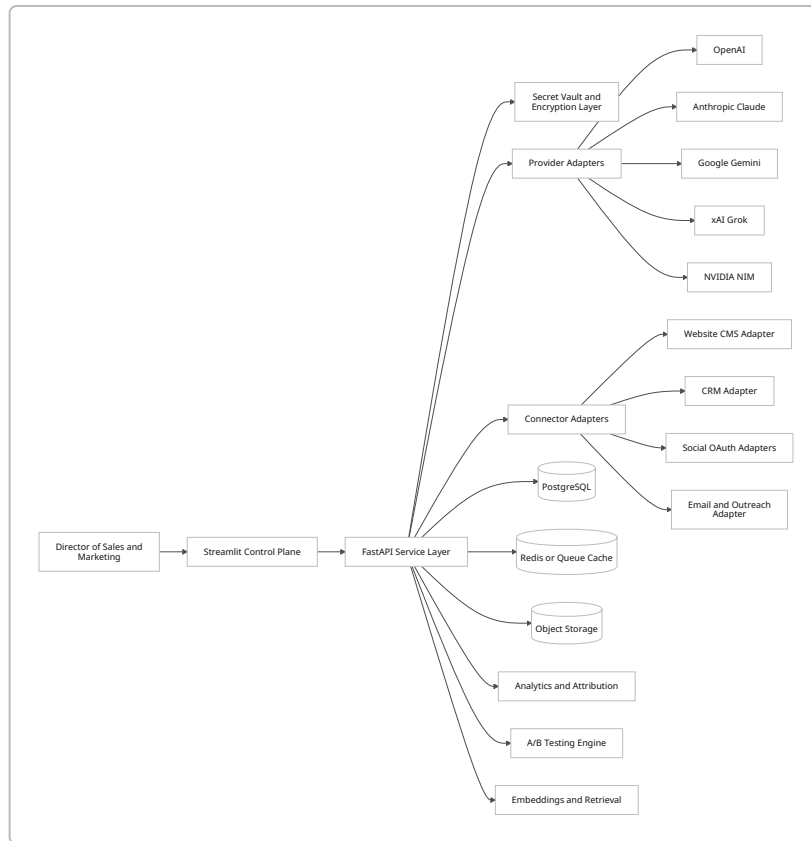
For NVIDIA NIM, the revenue-engine-relevant subset of the current catalog is worth calling out separately:

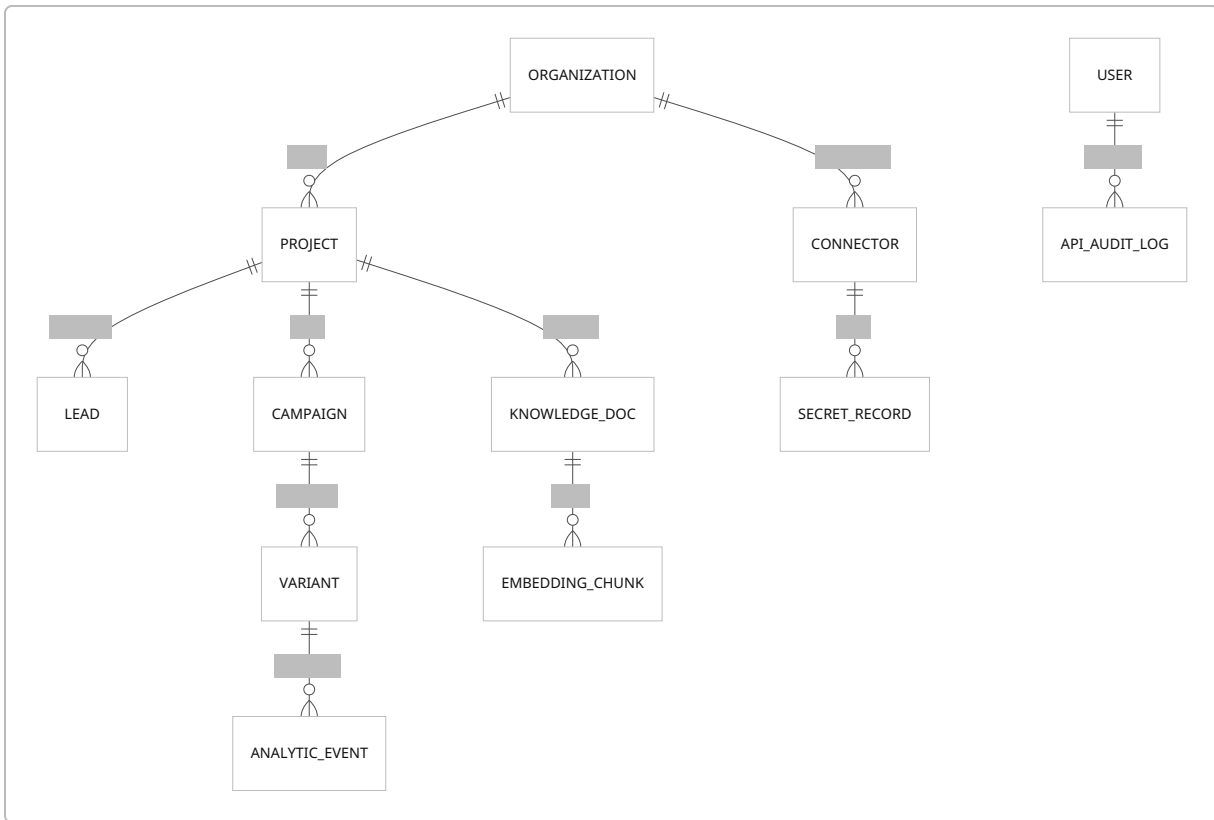
Category	Current relevant models and endpoints from official docs
LLM and reasoning	<code>z-ai/glm5.1</code> , <code>moonshotai/kimi-k2.6</code> , <code>deepseek-ai/deepseek-v4-flash</code> , <code>nvidia/nemotron-3-super-120b-a12b</code> , <code>nvidia/nemotron-3-ultra-550b-a55b</code> , <code>qwen/qwen3-5-122b-a10b</code> , <code>meta/llama-3.3-70b-instruct</code> and related LLM models appear on the LLM APIs reference. ¹⁸
Retrieval	<code>nvidia/nv-embed-v1</code> , <code>nvidia/nv-embedqa-e5-v5</code> , <code>nvidia/llama-nemotron-embed-1b-v2</code> , <code>nvidia/nv-rerankqa-mistral-4b-v3</code> , plus related NeMo Retriever models appear on the Retrieval APIs reference. ¹⁹
Vision and multimodal	<code>microsoft/phi-4-multimodal-instruct</code> , <code>meta/llama-3.2-11b-vision-instruct</code> , <code>meta/llama-3.2-90b-vision-instruct</code> , <code>meta/llama-4-maverick-17b-128e-instruct</code> , <code>google/paligemma</code> , and <code>nvidia/nemotron-nano-12b-v2-v1</code> appear on the Visual or Multimodal references. ²⁰
Image generation	<code>black-forest-labs/flux.1-schnell</code> , <code>stabilityai/stable-diffusion-3-medium</code> , and <code>stabilityai/stable-diffusion-xl</code> appear on NVIDIA's visual-model references, with at least SD3 documented under <code>ai.api.nvidia.com/v1/genai/...</code> . ²¹

The practical takeaway is simple: your engine should let the admin pick a **task class**—copywriting, persona analysis, long-form proposal generation, image generation, OCR/vision, embeddings, reranking, safety checks—and then let the backend map that class to the most appropriate provider and model. That is easier to operate, easier to swap, and safer than allowing each workflow to hard-code one provider forever. ²²

Master prompt and system architecture

The architecture below is the cleanest shape for your requirements.





The following is the **single enhanced master prompt** you can paste into another AI agent. It is intentionally strict, comprehensive, and designed to keep the build aligned with production security, sales outcomes, and admin usability.

You are a principal AI solutions architect, senior full-stack Python engineer, senior RevOps engineer, senior marketing automation architect, and security-conscious technical writer.

Your task is to design and generate a complete, production-ready “AI Automation Revenue Engine” for a Director of Sales & Marketing.

The operator will provide:

- website_link
- approved website admin credentials
- approved social platform login credentials or, preferably, app credentials / OAuth app details
- provider API keys as available
- optional CRM access
- optional project portfolio context

You must treat the following as UNSPECIFIED unless explicitly provided:

- CMS type
- hosting platform

- exact social platforms
- exact CRM
- analytics stack
- payment stack
- exact project count beyond "100+ projects available"
- budget ceiling

Core rules:

- Do not invent unspecified platforms.
- Detect, document, and preserve unknowns.
- Prefer official OAuth / scoped token workflows over raw password replay wherever the platform supports that.
- Use least privilege everywhere.
- Never hardcode secrets.
- Never commit secrets to source control.
- Build a modular architecture with provider adapters and connector adapters.
- Output only Python-first implementation with Streamlit frontend/control-plane and FastAPI backend/service layer.
- Include JavaScript helpers only where they improve UX inside the Streamlit or admin workflow.
- Make the admin UX easy, corporate, and non-technical to operate.

Primary objective:

Generate a full system that can connect, verify, orchestrate, and monetize digital sales and marketing assets from one admin console.

Mandatory capabilities:

1. Discovery and verification

- Accept website_link and determine likely CMS / stack / verification method.
- If CMS is unknown, perform safe discovery and document confidence level.
- Verify admin reachability without destructive actions.
- Verify connected social accounts using official token/app flows when possible.
- Produce a connector status report with health, permissions, and missing approvals.

2. Multi-provider AI layer

- Implement provider adapters for:
 - OpenAI
 - Anthropic Claude
 - Google Gemini
 - xAI Grok
 - NVIDIA NIM
- Normalize chat, structured output, embeddings, reranking, image generation, and safety tasks where applicable.
- Add routing policies by use case:
 - strategy / planning
 - copywriting
 - proposal generation

- multimodal analysis
- retrieval and rerank
- image generation
- Build fallback chains and timeout handling.

3. NVIDIA NIM integration

- Use current official NVIDIA docs and validate current live endpoints before coding.
- Support current LLM chat, embeddings/retrieval, and relevant image-generation endpoints.
- Include clear setup steps for NVIDIA API key generation from build.nvidia.com.
- Create an admin model catalog page that loads configured model IDs from a JSON config file, not from hardcoded UI constants.
- Mark model availability as "validated at deployment time".

4. Frontend and backend

- Frontend: Streamlit multipage operator console.
- Backend: FastAPI with modular routers and services.
- Include:
 - dashboard
 - providers settings
 - connector settings
 - lead pipeline
 - campaigns
 - experiments
 - analytics
 - knowledge base / embeddings
 - secrets / security page
 - audit logs
- Use a clean app layout and easy settings toggles.

5. Authentication and security

- Implement admin authentication with JWT-backed backend sessions.
- Store machine secrets in a vault abstraction.
- Encrypt sensitive secrets at rest.
- Support secret rotation workflows.
- Add audit logs for every credential change, provider change, and campaign action.
- Sanitize logs to avoid leaking secret values.
- Use HTTPS assumptions for production and document reverse-proxy requirements.

6. Revenue engine features

- lead capture forms
- landing page content generation
- CRM sync
- outreach sequence generation
- campaign scheduling
- A/B testing

- analytics and attribution
- recommendation engine for next best sales action
- monetization workflow hooks
- project portfolio showcase automation
- content repurposing from website assets into social and outreach assets

7. Embeddings and retrieval

- index site pages, FAQs, offers, product/service pages, sales collateral, and uploaded documents
- chunk content
- generate embeddings
- store vector metadata
- add reranking
- create reusable prompts for sales proposals, email outreach, meeting prep, campaign planning, and offer positioning

8. JavaScript helpers

- create lightweight helpers for:
 - copy to clipboard
 - OAuth popup / redirect handling
 - dynamic UTM builder
 - client-side validation
 - event tracking hooks
- keep JS minimal and isolated

9. Documentation and diagrams

Produce:

- executive summary
- architecture explanation
- data flow explanation
- entity relationship explanation
- Mermaid diagrams for architecture, data flow, and ERD
- deployment guide
- configuration guide
- test guide
- operations guide
- admin quick-start
- troubleshooting guide

10. Code output requirements

Generate a runnable repository with:

- frontend/app.py
- backend/main.py
- backend/routers/*
- backend/services/*
- backend/models/*
- backend/security/*
- config/settings.py

- tests/*
- requirements.txt
- .streamlit/config.toml example
- .env.example
- docker-compose.yml
- Makefile or simple run commands
- sample JSON configs
- curl examples
- sample API requests and responses

11. Testing

Include:

- FastAPI pytest tests
- Streamlit AppTest tests
- smoke tests
- connector health tests
- provider health tests
- sample curl commands
- failure-mode tests for missing secrets, expired tokens, and insufficient scopes

12. Cost and infra

- Provide a minimal production baseline infra suggestion
- Separate “essential now” from “optional later”
- Keep assumptions explicit
- Exclude any invented pricing
- Note variable LLM usage separately from fixed infrastructure

13. Output format

Output in this order:

- executive summary
- assumptions and unknowns
- repo tree
- architecture
- code
- config files
- testing
- deployment
- admin guide
- milestone plan
- deliverables checklist

Implementation details to prefer:

- Python 3.11+
- Streamlit
- FastAPI
- PostgreSQL
- Redis-compatible cache/queue

- object storage abstraction
- Pydantic settings
- httpx
- JWT auth
- typed service classes
- provider adapter interface
- connector adapter interface
- strong logging and structured error handling

NVIDIA and provider notes:

- Validate current provider docs before finalizing code.
- Do not assume all NVIDIA modalities share one base URL.
- Implement a provider registry and task-router.
- Keep model IDs configurable from JSON and environment.

Admin UX requirements:

- fully corporate look
- left sidebar navigation
- clean settings forms
- connection test buttons
- permission warnings
- onboarding wizard
- health indicators
- safe defaults
- minimal clicks

Final quality bar:

- production-minded
- secure by default
- runnable baseline
- well-commented
- easy to configure
- easy for a sales and marketing director to operate
- explicit about what is implemented now vs what remains connector-specific

That prompt is deliberately opinionated in the right places: it blocks the most dangerous shortcuts, forces explicit handling of unknowns, and makes the target AI produce both business deliverables and implementation artifacts.

Reference implementation scaffold

The scaffold below is a **runnable baseline** for the control plane and service layer. It is intentionally compact, but it already demonstrates the correct structure: JWT auth, encrypted secret storage, provider routing, lead capture, retrieval ingestion, and Streamlit orchestration. The fastest path is to use this as the initial repository skeleton, then let the AI agent expand connector-specific modules. The design aligns with current official Streamlit, FastAPI, and security guidance. ²³

Repository layout

```
ai_revenue_engine/  
├── backend/  
│   ├── main.py  
│   ├── security.py  
│   └── settings.py  
├── frontend/  
│   └── app.py  
├── tests/  
│   └── test_api.py  
├── .streamlit/  
│   └── config.toml  
├── requirements.txt  
└── .env.example
```

Backend service

```
# backend/main.py  
from datetime import datetime, timedelta, timezone  
from typing import Optional, Literal  
import base64  
import json  
import os  
import sqlite3  
import uuid  
  
import httpx  
import jwt  
from cryptography.fernet import Fernet  
from fastapi import FastAPI, HTTPException, Depends, Header  
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm  
from pydantic import BaseModel, HttpUrl  
  
DB_PATH = os.getenv("APP_DB_PATH", "engine.db")  
JWT_SECRET = os.getenv("JWT_SECRET", "change-me-in-production")  
JWT_ALG = "HS256"  
JWT_EXP_MIN = int(os.getenv("JWT_EXP_MIN", "30"))  
FERNET_KEY = os.getenv("FERNET_KEY", Fernet.generate_key().decode())  
  
fernet = Fernet(FERNET_KEY.encode())  
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/token")  
app = FastAPI(title="AI Revenue Engine")  
  
def db():
```

```

conn = sqlite3.connect(DB_PATH)
conn.execute("""
CREATE TABLE IF NOT EXISTS users (
    id TEXT PRIMARY KEY,
    username TEXT UNIQUE,
    password TEXT
)""")
conn.execute("""
CREATE TABLE IF NOT EXISTS secrets (
    id TEXT PRIMARY KEY,
    name TEXT UNIQUE,
    value BLOB NOT NULL,
    created_at TEXT NOT NULL
)""")
conn.execute("""
CREATE TABLE IF NOT EXISTS leads (
    id TEXT PRIMARY KEY,
    email TEXT,
    name TEXT,
    source TEXT,
    status TEXT,
    meta_json TEXT,
    created_at TEXT NOT NULL
)""")
conn.execute("""
CREATE TABLE IF NOT EXISTS kb_docs (
    id TEXT PRIMARY KEY,
    title TEXT,
    content TEXT,
    created_at TEXT NOT NULL
)""")
conn.commit()
return conn

def seed_admin():
    conn = db()
    cur = conn.execute("SELECT username FROM users WHERE username = ?",
("admin",))
    if not cur.fetchone():
        # Replace with hashed password in production. This is a bootstrap only.
        conn.execute(
            "INSERT INTO users (id, username, password) VALUES (?, ?, ?)",
            (str(uuid.uuid4()), "admin", os.getenv("BOOTSTRAP_ADMIN_PASSWORD",
"admin123!change")))
        conn.commit()
    conn.close()

```

```

seed_admin()

class TokenResponse(BaseModel):
    access_token: str
    token_type: str = "bearer"

class SecretIn(BaseModel):
    name: str
    value: str

class LeadIn(BaseModel):
    email: str
    name: Optional[str] = None
    source: str = "manual"
    status: str = "new"
    meta: dict = {}

class VerifyIn(BaseModel):
    website_link: HttpUrl
    cms_hint: Optional[str] = None

class ChatIn(BaseModel):
    provider: Literal["openai", "xai", "nvidia_llm"]
    model: str
    system: str = "You are a helpful revenue operations copilot."
    prompt: str

class EmbedIn(BaseModel):
    provider: Literal["openai", "nvidia_embed"]
    model: str
    input: list[str] | str
    input_type: Optional[Literal["query", "passage"]] = None

def create_token(sub: str) -> str:
    payload = {
        "sub": sub,
        "exp": datetime.now(timezone.utc) + timedelta(minutes=JWT_EXP_MIN),
    }
    return jwt.encode(payload, JWT_SECRET, algorithm=JWT_ALG)

def current_user(token: str = Depends(oauth2_scheme)) -> str:
    try:
        payload = jwt.decode(token, JWT_SECRET, algorithms=[JWT_ALG])
        return payload["sub"]
    except Exception:
        raise HTTPException(status_code=401, detail="Invalid token")

def save_secret(name: str, value: str):

```

```

    enc = fernet.encrypt(value.encode())
    conn = db()
    conn.execute(
        "INSERT OR REPLACE INTO secrets (id, name, value, created_at) VALUES
        (?, ?, ?, ?)",
        (str(uuid.uuid4()), name, enc, datetime.utcnow().isoformat())
    )
    conn.commit()
    conn.close()

def get_secret(name: str) -> str:
    conn = db()
    cur = conn.execute("SELECT value FROM secrets WHERE name = ?", (name,))
    row = cur.fetchone()
    conn.close()
    if not row:
        raise HTTPException(status_code=400, detail=f"Missing secret: {name}")
    return fernet.decrypt(row[0]).decode()

@app.get("/health")
def health():
    return {"ok": True, "service": "ai-revenue-engine"}

@app.post("/auth/token", response_model=TokenResponse)
def login(form: OAuth2PasswordRequestForm = Depends()):
    conn = db()
    cur = conn.execute(
        "SELECT username, password FROM users WHERE username = ?",
        (form.username,)
    )
    row = cur.fetchone()
    conn.close()
    if not row or form.password != row[1]:
        raise HTTPException(status_code=401, detail="Bad credentials")
    return TokenResponse(access_token=create_token(row[0]))

@app.post("/secrets")
def put_secret(payload: SecretIn, user: str = Depends(current_user)):
    save_secret(payload.name, payload.value)
    return {"ok": True, "name": payload.name, "updated_by": user}

@app.post("/integrations/verify")
async def verify(payload: VerifyIn, user: str = Depends(current_user)):
    async with httpx.AsyncClient(timeout=20) as client:
        resp = await client.get(str(payload.website_link))
    body = resp.text.lower()
    cms = payload.cms_hint or (
        "wordpress" if "wp-content" in body else

```

```

        "shopify" if "shopify" in body else
        "webflow" if "webflow" in body else
        "unknown"
    )
    return {
        "ok": resp.status_code < 400,
        "status_code": resp.status_code,
        "detected_cms": cms,
        "next_action": "Prefer platform-native token/OAuth connector over
password automation"
    }

@app.post("/leads")
def create_lead(payload: LeadIn, user: str = Depends(current_user)):
    conn = db()
    lead_id = str(uuid.uuid4())
    conn.execute(
        "INSERT INTO leads (id, email, name, source, status, meta_json,
created_at) VALUES (?, ?, ?, ?, ?, ?, ?)",
        (lead_id, payload.email, payload.name, payload.source, payload.status,
json.dumps(payload.meta), datetime.utcnow().isoformat())
    )
    conn.commit()
    conn.close()
    return {"ok": True, "lead_id": lead_id}

@app.get("/leads")
def list_leads(user: str = Depends(current_user)):
    conn = db()
    cur = conn.execute("SELECT id, email, name, source, status, meta_json,
created_at FROM leads ORDER BY created_at DESC")
    rows = cur.fetchall()
    conn.close()
    return [
        {
            "id": r[0], "email": r[1], "name": r[2], "source": r[3],
            "status": r[4], "meta": json.loads(r[5] or "{}"), "created_at": r[6]
        }
        for r in rows
    ]

@app.post("/knowledge")
def add_knowledge(title: str, content: str, user: str = Depends(current_user)):
    conn = db()
    doc_id = str(uuid.uuid4())
    conn.execute(
        "INSERT INTO kb_docs (id, title, content, created_at) VALUES
(?, ?, ?, ?)",

```



```

        (doc_id, title, content, datetime.utcnow().isoformat())
    )
    conn.commit()
    conn.close()
    return {"ok": True, "doc_id": doc_id}

@app.post("/providers/chat")
async def provider_chat(payload: ChatIn, user: str = Depends(current_user)):
    if payload.provider == "openai":
        url = "https://api.openai.com/v1/responses"
        key = get_secret("OPENAI_API_KEY")
        headers = {"Authorization": f"Bearer {key}"}
        body = {
            "model": payload.model,
            "input": [
                {"role": "system", "content": payload.system},
                {"role": "user", "content": payload.prompt},
            ],
        }

    elif payload.provider == "xai":
        url = "https://api.x.ai/v1/responses"
        key = get_secret("XAI_API_KEY")
        headers = {"Authorization": f"Bearer {key}"}
        body = {
            "model": payload.model,
            "input": [
                {"role": "system", "content": payload.system},
                {"role": "user", "content": payload.prompt},
            ],
        }

    elif payload.provider == "nvidia_llm":
        url = "https://integrate.api.nvidia.com/v1/chat/completions"
        key = get_secret("NVIDIA_API_KEY")
        headers = {"Authorization": f"Bearer {key}"}
        body = {
            "model": payload.model,
            "messages": [
                {"role": "system", "content": payload.system},
                {"role": "user", "content": payload.prompt},
            ],
        }
    else:
        raise HTTPException(status_code=400, detail="Unsupported provider")

    headers["Content-Type"] = "application/json"
    async with httpx.AsyncClient(timeout=60) as client:

```

```

        r = await client.post(url, headers=headers, json=body)
    if r.status_code >= 400:
        raise HTTPException(status_code=r.status_code, detail=r.text)

    data = r.json()
    # Normalize rough output
    text = (
        data.get("output_text")
        or data.get("choices", [{}])[0].get("message", {}).get("content")
        or data
    )
    return {"ok": True, "provider": payload.provider, "model": payload.model,
            "text": text}

@app.post("/providers/embed")
async def provider_embed(payload: EmbedIn, user: str = Depends(current_user)):
    if payload.provider == "openai":
        url = "https://api.openai.com/v1/embeddings"
        key = get_secret("OPENAI_API_KEY")
        headers = {"Authorization": f"Bearer {key}", "Content-Type":
"application/json"}
        body = {"model": payload.model, "input": payload.input}

    elif payload.provider == "nvidia_embed":
        url = "https://integrate.api.nvidia.com/v1/embeddings"
        key = get_secret("NVIDIA_API_KEY")
        headers = {"Authorization": f"Bearer {key}", "Content-Type":
"application/json"}
        body = {"model": payload.model, "input": payload.input}
        if payload.input_type:
            body["input_type"] = payload.input_type
    else:
        raise HTTPException(status_code=400, detail="Unsupported embedding
provider")

    async with httpx.AsyncClient(timeout=60) as client:
        r = await client.post(url, headers=headers, json=body)
    if r.status_code >= 400:
        raise HTTPException(status_code=r.status_code, detail=r.text)
    return {"ok": True, "provider": payload.provider, "data": r.json()}

```

Streamlit control plane

```

# frontend/app.py
import os
import requests

```

```

import streamlit as st
import streamlit.components.v1 as components

API = os.getenv("BACKEND_URL", "http://localhost:8000")

st.set_page_config(page_title="AI Revenue Engine", layout="wide")
st.title("AI Automation Revenue Engine")

def auth_headers():
    token = st.session_state.get("token")
    return {"Authorization": f"Bearer {token}"} if token else {}

with st.sidebar:
    st.header("Admin")
    if "token" not in st.session_state:
        with st.form("login"):
            u = st.text_input("Username", value="admin")
            p = st.text_input("Password", type="password")
            if st.form_submit_button("Login"):
                r = requests.post(f"{API}/auth/token", data={"username": u,
"password": p})
                if r.ok:
                    st.session_state["token"] = r.json()["access_token"]
                    st.success("Logged in")
                else:
                    st.error(r.text)
            else:
                st.success("Authenticated")
                if st.button("Log out"):
                    st.session_state.clear()
                    st.rerun()

tabs = st.tabs([
    "Overview", "Connectors", "Providers", "Leads", "Knowledge", "AI Workspace"
])

with tabs[0]:
    st.subheader("System health")
    r = requests.get(f"{API}/health")
    st.json(r.json())

with tabs[1]:
    st.subheader("Website verification")
    website = st.text_input("Website link")
    cms_hint = st.text_input("CMS hint", placeholder="optional")
    if st.button("Verify website"):
        r = requests.post(
            f"{API}/integrations/verify",

```

```

        headers=auth_headers(),
        json={"website_link": website, "cms_hint": cms_hint or None},
    )
    st.json(r.json())

with tabs[2]:
    st.subheader("Provider secrets")
    secret_name = st.selectbox(
        "Secret",
        ["OPENAI_API_KEY", "XAI_API_KEY", "NVIDIA_API_KEY"]
    )
    secret_value = st.text_input("Secret value", type="password")
    if st.button("Save secret"):
        r = requests.post(
            f"{API}/secrets",
            headers=auth_headers(),
            json={"name": secret_name, "value": secret_value},
        )
        st.json(r.json())

with tabs[3]:
    st.subheader("Lead capture")
    with st.form("lead"):
        email = st.text_input("Email")
        name = st.text_input("Name")
        source = st.text_input("Source", value="landing_page")
        if st.form_submit_button("Create lead"):
            r = requests.post(
                f"{API}/leads",
                headers=auth_headers(),
                json={"email": email, "name": name, "source": source, "meta":
{}}},
            )
            st.json(r.json())
    if st.button("Refresh leads"):
        r = requests.get(f"{API}/leads", headers=auth_headers())
        st.dataframe(r.json(), use_container_width=True)

with tabs[4]:
    st.subheader("Knowledge ingestion")
    title = st.text_input("Document title")
    content = st.text_area("Document content", height=200)
    if st.button("Store knowledge"):
        r = requests.post(
            f"{API}/knowledge",
            headers=auth_headers(),
            params={"title": title, "content": content},
        )

```

```

        st.json(r.json())

with tabs[5]:
    st.subheader("AI workspace")
    provider = st.selectbox("Provider", ["openai", "xai", "nvidia_llm"])
    model = st.text_input("Model", value="gpt-5.5")
    system = st.text_area("System prompt",
value="You are a revenue optimization copilot.")
    prompt = st.text_area("User prompt", height=160)
    if st.button("Run"):
        r = requests.post(
            f"{API}/providers/chat",
            headers=auth_headers(),
            json={"provider": provider, "model": model, "system": system,
"prompt": prompt},
        )
        st.json(r.json())

components.html("""
<script>
function copyText(text){
    navigator.clipboard.writeText(text);
}
window.addEventListener("message", (event) => {
    if (event.data && event.data.type === "copy") copyText(event.data.value);
});
</script>
""", height=0)

```

Streamlit corporate theme

```

# .streamlit/config.toml
[client]
showErrorDetails = "none"
toolbarMode = "minimal"

[server]
headless = true
cookieSecret = "replace-with-shared-cookie-secret"

[theme]
base = "light"
primaryColor = "#0F4C81"
backgroundColor = "#F8FAFC"

```

```
secondaryBackgroundColor = "#FFFFFF"  
textColor = "#0F172A"
```

Environment example

```
# .env.example  
BACKEND_URL=http://localhost:8000  
APP_DB_PATH=engine.db  
JWT_SECRET=replace_with_long_random_secret  
JWT_EXP_MIN=30  
FERNET_KEY=replace_with_fernet_key  
BOOTSTRAP_ADMIN_PASSWORD=replace_me
```

Requirements

```
fastapi  
uvicorn  
httpx  
pydantic  
pyjwt  
cryptography  
python-multipart  
streamlit  
requests  
pytest
```

Tests

```
# tests/test_api.py  
from fastapi.testclient import TestClient  
from backend.main import app  
  
client = TestClient(app)  
  
def test_health():  
    r = client.get("/health")  
    assert r.status_code == 200  
    assert r.json()["ok"] is True  
  
def test_login_and_lead_flow():  
    token = client.post("/auth/token", data={"username": "admin", "password":  
"admin123!change"}).json()["access_token"]  
    headers = {"Authorization": f"Bearer {token}"}
```

```

lead = client.post("/leads", headers=headers, json={
    "email": "demo@example.com",
    "name": "Demo Lead",
    "source": "pytest",
    "meta": {"campaign": "smoke"}}
))
assert lead.status_code == 200

all_leads = client.get("/leads", headers=headers)
assert all_leads.status_code == 200
assert any(item["email"] == "demo@example.com" for item in all_leads.json())

```

cURL examples

OpenAI and xAI use Responses-style flows; NVIDIA LLM chat and embeddings use the NIM endpoints documented in the official docs. The following examples are aligned with the current documented patterns.

24

```

# Login to backend
curl -X POST http://localhost:8000/auth/token \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "username=admin&password=admin123!change"

# Save NVIDIA key
curl -X POST http://localhost:8000/secrets \
  -H "Authorization: Bearer YOUR_BACKEND_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name":"NVIDIA_API_KEY","value":"YOUR_NVIDIA_KEY"}'

# Verify website
curl -X POST http://localhost:8000/integrations/verify \
  -H "Authorization: Bearer YOUR_BACKEND_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"website_link":"https://example.com","cms_hint":null}'

# Run NVIDIA LLM chat
curl -X POST http://localhost:8000/providers/chat \
  -H "Authorization: Bearer YOUR_BACKEND_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "provider":"nvidia_llm",
    "model":"z-ai/glm5.1",
    "system":"You are a B2B marketing strategist.",
    "prompt":"Create a three-email outbound sequence for CFO personas."
  }'

```

```
# Run NVIDIA embeddings
curl -X POST http://localhost:8000/providers/embed \
  -H "Authorization: Bearer YOUR_BACKEND_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "provider":"nvidia_embed",
    "model":"nvidia/nv-embed-v1",
    "input":["Revenue engine playbook", "Lead scoring rubric"],
    "input_type":"passage"
  }'
```

Deployment, infrastructure, costs, and testing

A clean low-end production deployment for this stack is: **one Streamlit web service, one FastAPI private service, one PostgreSQL instance, and one Redis-compatible cache or queue.** Streamlit's configuration system supports production settings, cookie secrets, and theming; FastAPI supports service testing and JWT auth; OWASP guidance supports moving secrets out of the repo and into centralized stores with automation and least privilege. ²⁵

A practical deployment sequence is:

1. Put all secrets into a non-git secret manager or secure environment injection path. Streamlit explicitly recommends not storing unencrypted secrets in a repository and supports `st.secrets` or external secret managers; Streamlit also notes secret changes may require a restart. ²⁶
2. Set a shared `server.cookieSecret` across Streamlit replicas if you ever scale horizontally. ²⁷
3. Use FastAPI JWT auth with a long random signing key; the official tutorial demonstrates the core token flow and token expiry pattern. ²⁸
4. For human passwords, follow NIST's current guidance: minimum 15 characters when used as a single factor, allow long passwords, avoid composition-rule theater, and do not force arbitrary periodic changes. For machine credentials, follow OWASP and automate rotation. This distinction matters. ²⁹
5. Register social and CRM apps separately for development, staging, and production, and use exact callback URLs over HTTPS in production. X documents this explicitly, and Pinterest and HubSpot follow the same general OAuth discipline. ³⁰
6. For NVIDIA NIM, generate an API key from the build portal, verify free serverless availability for the selected model cards, and configure task-specific endpoints instead of a single universal base URL. ³¹

The table below is a conservative low-end cost sketch using official public pricing for infrastructure only. It excludes variable LLM token spend, ad spend, domain registration, and any premium CRM or social platform plans.

Component	Example low-end option	Current public price signal	Notes
Streamlit UI service	Render Starter web service	\$7/month. ³²	Good starting point for the control plane.

Component	Example low-end option	Current public price signal	Notes
FastAPI backend service	Render Starter private or web service	\$7/month. ³²	Keep backend private if fronted internally.
PostgreSQL	Render Basic-256MB Postgres	\$6/month. ³³	Entry-level managed DB.
Redis-compatible cache	Render Key Value Starter	\$10/month. ³³	Or substitute Upstash.
Redis alternative	Upstash Fixed 250MB	\$10/month, or pay-as-you-go at \$0.20 per 100K commands. ³⁴	Useful if you want lower idle cost or lighter queueing.
Disk and storage	Render persistent disk	\$0.25/GB/month. ³⁵	Only if you need attached storage beyond DB/object storage.
NVIDIA development inference	NVIDIA build portal free serverless APIs for development	Advertised as free for development on supported model cards. ³⁶	Re-validate live model availability.

That means a realistic entry baseline is roughly **\$20–\$30/month for core infrastructure** before any paid model usage, rising to **\$30–\$40/month** if you keep a dedicated Redis tier instead of using a lighter pay-as-you-go option. The expensive part later is typically not the app shell; it is the external platforms, token usage, experiment traffic, and ads.

For testing, you should keep two test layers from day one. Streamlit’s official testing guide supports `pytest` plus `streamlit.testing.v1.AppTest`, and FastAPI’s official testing guide supports `TestClient` with standard `pytest` assertions. ³⁷

Test layer	What to validate	Tooling
UI smoke tests	Login flow, page rendering, settings forms, lead creation, AI workspace interactivity	Streamlit <code>AppTest</code> + <code>pytest</code> . ³⁸
API unit and integration tests	JWT issuance, secret writes, connector verification, lead CRUD, provider routing	FastAPI <code>TestClient</code> + <code>pytest</code> . ³⁹
Connector health tests	Missing scopes, expired tokens, invalid callback URLs, bad credentials	Provider-specific fixture tests
Provider health tests	OpenAI, xAI, NVIDIA connectivity and timeout handling	Scheduled smoke jobs
Security tests	Secret redaction, unauthorized access, role boundaries, audit logging completeness	Negative-path API tests

Implementation checklist and open questions

The milestone plan below is the most defensible way to execute without overbuilding too early.

Milestone	Deliverables	Exit criteria
Foundation	Repo scaffold, Streamlit console, FastAPI backend, JWT auth, encrypted secret storage, health checks	Local dev stack boots and passes smoke tests
Connector discovery	Website verification, CMS detection, token-first connector policy, connector health dashboard	Website can be safely profiled and verified
Provider layer	OpenAI, xAI, NVIDIA working adapters; Anthropic and Gemini adapter shells with documented validation path	Multi-provider routing works for core tasks
Revenue workflows	Lead capture, knowledge ingestion, CRM adapter framework, outreach templates, experiment entities	Operator can move from lead ingestion to campaign draft
Optimization	Embeddings, rerank, attribution events, A/B testing workflows, analytics summaries	At least one closed-loop experiment can run
Hardening	Secret rotation workflows, audit logs, environment separation, backup strategy, production deployment	Ready for controlled pilot launch

The admin UX should stay intentionally boring and corporate. Concretely, that means a left sidebar, a short onboarding wizard, clear health badges, “Test connection” buttons, safe defaults, and branded theming via `.streamlit/config.toml`. Streamlit’s theming docs explicitly support light and dark themes, separate sidebar/main-body styling, primary color, font, border radius, and background controls, which is enough to give this a polished internal-product feel without abandoning Streamlit. ⁴⁰

A short implementation checklist for the build agent:

- Create separate dev, staging, and prod apps for any OAuth-driven connector. ¹⁴
- Move all provider keys and connector secrets into encrypted storage immediately. ⁴¹
- Use tokenized connectors instead of standing browser logins whenever possible. ⁵
- Treat NVIDIA modalities separately during configuration. ⁴²
- Keep model IDs and routing policies in JSON or environment-driven config, not UI constants. ⁴³
- Add Streamlit and FastAPI tests before adding more connectors. ⁴⁴
- Hide detailed server errors in production UI and keep logs sanitized. ⁴⁵

Open questions and limitations:

- **CMS type is unspecified**, so the final connector implementation must remain adapter-based.
- **Social platforms are unspecified**, so the system should not hard-code Meta, LinkedIn, X, Pinterest, or TikTok until the operator confirms the target set.

- **CRM is unspecified**; HubSpot is the easiest default reference, but the architecture should remain CRM-agnostic. ¹³
- **Budget is unspecified**, so the cost table above is an infrastructure-only starting point, not a full TCO forecast.
- A fixed official **“77 NVIDIA model endpoint names”** artifact could not be corroborated against the current official catalog; the current official model inventory is broader, so the safer production choice is to generate the JSON catalog from the live docs at build or deployment time rather than to trust an older static list. ⁴⁶

¹ ⁵ <https://developers.hubspot.com/docs/apps/developer-platform/build-apps/authentication/oauth/working-with-oauth>

<https://developers.hubspot.com/docs/apps/developer-platform/build-apps/authentication/oauth/working-with-oauth>

² ⁸ ²⁴ <https://platform.openai.com/docs/api-reference/introduction>

<https://platform.openai.com/docs/api-reference/introduction>

³ ²¹ ⁴⁶ <https://docs.api.nvidia.com/nim/reference/models-1>

<https://docs.api.nvidia.com/nim/reference/models-1>

⁴ ⁷ ¹⁶ ²³ ²⁵ ²⁶ ⁴¹ <https://docs.streamlit.io/develop/concepts/connections/secrets-management>

<https://docs.streamlit.io/develop/concepts/connections/secrets-management>

⁶ https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

⁹ <https://docs.anthropic.com/en/api/getting-started>

<https://docs.anthropic.com/en/api/getting-started>

¹⁰ ²² <https://ai.google.dev/gemini-api/docs>

<https://ai.google.dev/gemini-api/docs>

¹¹ <https://docs.x.ai/docs/overview>

<https://docs.x.ai/docs/overview>

¹² ¹⁸ ⁴³ <https://docs.api.nvidia.com/nim/reference/llm-apis>

<https://docs.api.nvidia.com/nim/reference/llm-apis>

¹³ <https://developers.hubspot.com/docs/api/overview>

<https://developers.hubspot.com/docs/api/overview>

¹⁴ ³⁰ <https://docs.x.com/fundamentals/developer-apps>

<https://docs.x.com/fundamentals/developer-apps>

¹⁵ <https://developers.pinterest.com/docs/getting-started/introduction/>

<https://developers.pinterest.com/docs/getting-started/introduction/>

¹⁷ ²⁸ <https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/>

<https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/>

¹⁹ <https://docs.api.nvidia.com/nim/reference/retrieval-apis>

<https://docs.api.nvidia.com/nim/reference/retrieval-apis>

²⁰ <https://docs.api.nvidia.com/nim/reference/visual-models-apis>

<https://docs.api.nvidia.com/nim/reference/visual-models-apis>

27 45 <https://docs.streamlit.io/develop/api-reference/configuration/config.toml>

<https://docs.streamlit.io/develop/api-reference/configuration/config.toml>

29 <https://pages.nist.gov/800-63-4/sp800-63b.html>

<https://pages.nist.gov/800-63-4/sp800-63b.html>

31 36 <https://build.nvidia.com/explore/discover>

<https://build.nvidia.com/explore/discover>

32 33 35 <https://render.com/pricing>

<https://render.com/pricing>

34 <https://upstash.com/pricing/redis>

<https://upstash.com/pricing/redis>

37 38 44 <https://docs.streamlit.io/develop/concepts/app-testing/get-started>

<https://docs.streamlit.io/develop/concepts/app-testing/get-started>

39 <https://fastapi.tiangolo.com/tutorial/testing/>

<https://fastapi.tiangolo.com/tutorial/testing/>

40 <https://docs.streamlit.io/develop/concepts/configuration/theming>

<https://docs.streamlit.io/develop/concepts/configuration/theming>

42 https://docs.api.nvidia.com/nim/reference/abacusai-dracarys-llama-3_1-70b-instruct-infer

https://docs.api.nvidia.com/nim/reference/abacusai-dracarys-llama-3_1-70b-instruct-infer